

Combinatorial Optimization – project

1. Authors:

- a) Kuba Czech, 156 035
- b) Adam Wilczyński, 156 065

2. Theoretical part – 5 minutes is quite a lot of time. So we came to the conclusion that we will try to implement two algorithms. Each producing two solutions and at the end better one will be chosen and printed (unless time is exceeded, then probably the first one will be printed).

The first approach is greedy, it was selected because it is an efficient algorithm. In short period of time it can produce good solution (can be suboptimal but still it is likely that quality of this solution will be high).

The second approach (genetic algorithm with additional help of greedy and dynamic programming) was selected, because greedy will produce suboptimal order of libraries in short period of time. Then dynamic programming will allow us to improve the greedy solution a bit, as we want to pack some libraries from left ones, which will let us scan as many books as possible in the remaining period of time. In result the genetic algorithm produces high quality solutions by performing crossover and mutation so it is suitable for this project.

3. Implementation part:

In the first (only greedy) approach, we sort libraries from best to worst one. Then, in each iteration we sign up the best library which is left and select proper amount of the best books which are in this library (proper amount means as many as possible given that we have some deadline). In the same time, we are deleting chosen books from other libraries to avoid choosing a book more than one time (we set availability of this book in all libraries to 0). We repeat this process until signing up next library will be done after deadline.

Here several numpy arrays were used because even though np.array has some limitations it is more efficient than python list (faster and takes less memory). We have separate np.arrays for signup time, number of books possible to scan per day etc. Also, one big 2D (nr of books x nr of libraries) to store availability of books in each library as a boolean value (True if book b is in library l, False otherwise). Solution was composed of two lists – one storing order of libraries, second one (2D) storing order of books for given library. Here np.array was not the best idea because we didn't know final length solution and size of 2D list.

The second approach is based on the concept of genetic algorithm. Here we need to answer 3 questions:

1. Which library to sign up?
2. In which order sign up chosen in point 1. libraries?
3. Which books to scan in which library

And that's how this approach works. First, we pick greedily libraries based on some coefficient. It measures how good is the library given shipping count per day, books it can scan per day, signup time, how valuable are books stored there. We want to minimize signup time and maximize everything else.

Now, next library will be signed up after deadline, but it is possible that there will be still some days to sign some libraries which are left (we will call this difference *delta*). To solve this problem and maximize score, we will use dynamic programming, where weight will be signup time and value will be a number of books it can scan per day. We sort books (by number of books to scan per day) selected by use of dynamic programming and append them to books selected greedily.

After steps above, we have selected libraries and its order. Now we will optimize score by choosing which books to scan in which libraries (thanks to genetic algorithm). We have order of library and as initial population we will select books in each library at random (without replacement). Evaluation function is total score.

Crossover works as follows: we have two solutions and select the index of breaking point at random, then we take part of first solution from 0 to index-1 and from second solution from index to the end. It is important to notice that it works because crossover won't change order of libraries because order is fixed from the beginning.

Mutation for some solution works as follows: we will randomly pick a library in this solution and select a new order of books (it may be a permutation or new books may appear) and repeat the process for 25% of libraries. Genetic algorithm works in normal way i.e. creates an initial population, then for 200 generations (or until there is no improvement 10 times in a row) it produces offspring (crossover and mutation), evaluates offspring, makes selection and repeats the process.

Here np.arrays were combined with lists, i.e., arrays were storing known information (about books, signup times etc.). The solution from the genetic algorithm was stored in a 1D list and 2D list (again, we don't know the final length of the solution).

4. Bibliography:

- a) Laboratories: 1 – 8
- b) Wikipedia.org
- c) "Principled Modeling of the Google Hash Code Problems for Meta-Heuristics", Pedro Miguel Duque Rodrigues